

Introduction of 'C' Language

One of the most widely used programming languages on the planet



Fast

Extremely quick compared to other programming languages



Flexible

Can be utilized in both applications and technology



Foundation

Learning C makes other languages easier to learn

Introduction to C Language 🚀

C is a general-purpose programming language that was created in 1972 and is still widely used today.

Key Characteristics 🔑

Performance



C is extremely quick when compared to other programming languages such as Java and Python.

Flexibility



C is a very flexible programming language that may be utilized in both applications and technology.

Syntax Comparable



If you know C, you will have no trouble learning other popular programming languages such as Java, Python, C++, C#, and so on.

```
// Simple C program demonstrating its syntax
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
}
```

```
    return 0;  
}
```



Real-World Applications

C is quite powerful; it has been used to create systems for operating systems, databases, applications, and so on. Many modern systems still rely on C for performance-critical components.

2/9

History of C Language

The creation of C is inextricably linked to the development of the Unix operating system.

Early Development

Unix was first developed in assembly language on a PDP-7 by Dennis Ritchie and Ken Thompson, who included other ideas from colleagues. They eventually opted to move the operating system to the PDP-11.

```
// Thompson's journey to creating B (precursor to C)  
  
// 1. Attempted to create a Fortran compiler but abandoned it  
// 2. Designed BCPL, a simplified version of systems programming language  
// 3. Adjusted syntax to be less complex and more comparable to SMALGOL  
// 4. Thompson dubbed the result B  
  
// B was described as "BCPL semantics with an abundance of SMALGOL syntax"
```



Limitations of B

B was too sluggish and couldn't make use of PDP-11 characteristics like byte addressability, so few utilities were eventually created in it. This limitation led to the development of C.

3/9

First C and New B Launch

In 1971, Ritchie enhanced B to use PDP-11, introducing New B (NB) as a character data type.

Evolution from B to C

1

New B (NB) - Introduced character data type to utilize PDP-11 features

2

Unix Kernel Design - Thompson used NB to design the Unix kernel

3

Language Features - NB introduced arrays of int and char, pointers, arrays of all types, and function return types

4

Renaming to C - Arrays within expressions were converted to pointers, and the language was renamed C

```
// Example of early C syntax showing array-pointer relationship
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int *ptr = arr; // Array name converted to pointer

    // Both expressions access the same element
    printf("arr[2] = %d\n", arr[2]);
    printf("*(ptr+2) = %d\n", *(ptr+2));

    return 0;
}
```



Version 2 Unix

Version 2 Unix, commonly known as Research Unix, contained the C compiler and several tools written with it. This marked the beginning of C's widespread adoption in systems programming.

4/9

Mechanisms and Modifications of the Unix Kernel

The Unix kernel was largely re-implemented in C in Version 4 Unix, which was published in November 1973.

Significant Features Added



Struct Types

Introduced in Version 4 Unix

Preprocessor

Created in 1973 at Alan Snyder's request



File Inclusion

Inspired by BCPL and PL/I techniques

```
// Example of struct types introduced in Version 4 Unix
#include <stdio.h>

// Struct type - a significant addition to C
struct Process {
    int pid;
    char name[50];
    int priority;
};

int main() {
    struct Process p1 = {1234, "init", 1};
    printf("Process: %s, PID: %d, Priority: %d\n", p1.name, p1.pid, p1.priority);
    return 0;
}
```



Preprocessor Evolution

The initial preprocessor form simply contained files and basic string replacements: parameter less macro `#include` and `#define`. Soon after, it was expanded to include macros with arguments and

conditional compilation, mainly by Mike Lesk and subsequently by John Reiser.

5/9

K&R C and Standardization

Unix was among the first operating system kernels written in a language other than assembly.

Early Standardization Efforts

Ritchie and Stephen C. Johnson made additional improvements to the language in 1977 to improve the portability of the Unix operating system. Johnson's Portable C Compiler was the foundation for various C implementations on new platforms.

```
// Example of K&R C function declaration style
#include <stdio.h>

// K&R style function declaration

/*
int add(a, b)
int a;
int b;
{
return a + b;
}
*/

// Modern ANSI C style function declaration
int add(int a, int b) {
    return a + b;
}

int main() {
    printf("5 + 3 = %d\n", add(5, 3));
}
```

```
return 0;  
}
```

Language Extensions

Several features were added to the language in the years after its release:



Void functions - Functions that don't return a value



Struct return types - Functions that return struct or union kinds



Struct allocation - Struct data type allocation



Enumerated types - Named integer constants



Need for Standardization

The vast number of extensions and absence of agreement on a standard library, along with the popularity of the language and the fact that not even Unix compilers accurately implemented the K&R specification, made standardization necessary.

6/9

ANSI C and ISO C

In the late 1970s and 1980s, C was implemented for various mainframe computers, minicomputers, and microcomputers.

Standardization Process

In 1983, the American National Standards Institute (ANSI) formed a committee, X3J11, to establish a standard specification of C. The standard was ratified as ANSI X3.159-1989 "Programming Language C" in 1989, often referred to as ANSI C, Standard C, or sometimes C89.

```
// Example of ANSI C standard features
#include <stdio.h>
#include <stdlib.h>

// Function prototypes - a key ANSI C feature
int calculate(int x, int y);

int main() {
    int result = calculate(10, 20);
    printf("Result: %d\n", result);
    return 0;
}

int calculate(int x, int y) {
    return x * y + 5;
}
```

ISO Adoption

In 1990, the ANSI C standard was adopted by the International Organization for Standardization (ISO) as ISO/IEC 9899:1990, also known as C90. The C standardization process aimed to produce a superset of K&R C, incorporating unofficial features and additional features.

C89 is supported by current C compilers and most modern C code is based on it. This standardization helped establish C as a truly portable programming language across different platforms.

7/9

Evolution of C Standards

The C language has continued to evolve with several standard revisions over the years.

1999

C99

Introduced inline functions, data types, variable-length arrays, flexible array members, floating point support, variadic macros, and one-line comments.

2011

C11

Introduced type generic macros, anonymous structures, Unicode support, atomic operations, multi-threading, and bounds-checked functions.

2018

C17

The latest standard for C, published as ISO/IEC 9899:2018. It has no new language attributes, merely technical repairs and explanations of C11 flaws.

2024

C23

The informal moniker for the next main C language standard version (following C17). It is scheduled to be released in 2024.

```
// Example of C99 features
#include <stdio.h>

// One-line comments (C99 feature)
// Variable-length arrays (C99 feature) void process_array(int size) {
    int arr[size]; // VLA - size determined at runtime
    for (int i = 0; i < size; i++) {
        arr[i] = i * i;
    }
}

// Inline function (C99 feature)
inline int square(int x) {
    return x * x;
}
```

8/9

Embedded C

Embedded C has always required nonstandard additions to handle exotic features like fixed-point arithmetic, many different memory banks, and fundamental I/O operations.

Standardization of Embedded C

The C Standards Committee produced a technical report in 2008 that extended the C language to solve these difficulties by creating a uniform standard to which all implementations must comply.

```
// Example of Embedded C features
#include <stdio.h>

// Named address spaces (Embedded C feature)
/*
__far int far_variable; // Variable in far memory space
__near int near_variable; // Variable in near memory space
*/

// Fixed-point arithmetic (Embedded C feature)
/*
typedef _Accum fixed_point; // Fixed-point type
fixed_point a = 3.5k; // k suffix for fixed-point literals
fixed_point b = 2.0k;
fixed_point result = a * b;
*/

// Basic I/O hardware addressing (Embedded C feature)
/*
#define PORTA (*(volatile unsigned char *)0x1000)
#define DDRB (*(volatile unsigned char *)0x1004)

// Set PORTA bit 3
PORTA |= (1 << 3);
*/
```

Embedded C Features

Embedded C has features not found in standard C, including:



Fixed-point Arithmetic

For systems without floating-point hardware

Named Address Spaces

For accessing different memory banks



Hardware I/O Addressing

For direct hardware manipulation

Summary 🎉

C has evolved from a simple language for writing Unix utilities to a standardized, powerful programming language used across diverse platforms, from supercomputers to embedded systems. Its influence can be seen in many modern programming languages, and its continued evolution ensures it remains relevant in today's computing landscape.